Гайд по тому, как этим пользоваться

Автор Дидык Иван

Тут могло быть ещё какое-нибудь описание

Документ скомпилирован 2026-02-20
Версия typst 0.14.2

Содержание

Написание текста, текстовые комманды - - -

Написание текста, текстовые комманды

Блоки

Есть блоки для написания определений/теорем/лемм/... и заданий:

> Определение какой-то термин для примера

Доказательство или решение

> Название задачи для примера Задача

Доказательство или решение

Есть ещё горизонтальная черта:

И такая стрелочка для пометок:

↣ Это заметка. Тут могла быть объяснёна какая-то запись, которую все уже забыли с прошлого семестра.

Математика

Для неё я написал довольно много сокращений, потому что формулы в ванильном typst довольно медленно писать.

Стили букв

Двойные буквы в typst пишутся как `AA`, `BB`, ...

$$\mathbb{ABCD}...\mathbb{XYZ}$$

Подобным образом я сделал и калиграфические буквы как `cA`, `cB`, ...

$$\mathcal{ABCD}...\mathcal{XYZ}$$

Фрактур как `fA`, `fB`, ...:

$$\mathfrak{ABCD}...\mathfrak{XYZ}$$

Рукописные (не каллиграфические!) как `sA`, `sB`, ...

$$\mathscr{ABCD}...\mathscr{XYZ}$$

Сокращения

Все определены в `src/sym.typ`. Тут я перечисляю только самые ходовые.

Для линий сверху/снизу можно использовать `bar()`/`und()`.

$$\overline{\lim} \quad \underline{\lim}$$

Для скобок снизу/сверху есть `ub(content, annot)` (снизу) и `ob(content, annot)` (сверху). Содержимое подписи у них влияет на ширину элемента.

$$\underbrace{A \cup B}_{A \subset B,\ \text{так что} = B} \quad \setminus \overbrace{C}^{=\emptyset} = B$$

Должен быть вариант где не влияет, но это

todo:

Для интегралов есть сокращения `int`, `iint` и так до `iiiint` для кратных:

$$\iint_M (x + yz)\,\mathrm{d}x \wedge \mathrm{d}y$$

Для объединений пересечений `uu` и `nn`. Если нужен больной оператор, то `uuu` и `nnn`.

$$C = \bigcup_{i=1} (A_i \cap B_i)$$

Есть `hh` и `hhh` для пробелов. Если этих вариантов не хватает, то всегда есть `h(distance)`.

$$A \quad B$$
$$A \qquad B$$

Для сужений или подстановок есть `with()`, к которому используются субскрипты/супскрипты.

$$\int_1^2 x = \frac{x^2}{2}\bigg|_1^2 = \frac{3}{2} \qquad g = f|_M$$

Также ещё несколько разных удобных мелочей:

Произведение	`xx`	$\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$
Плюс минус	`pm`	$H = 3 \pm 6\ \text{см}$
Пустое множество	`void`	$A = \emptyset$
Симметричная разница	`sdiff`	$A = B \triangle C$
Подмножество	`sub`	$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$
Градиент	`grad`	∇f
Эпсилон (алиас `epsilon`)	`eps`	$\forall \varepsilon\ \exists \delta \ldots$
Угловые скобки	`shell()`	$A_1 = \langle v_{11}, v_{12} \rangle$
Индикатор	`d1`	$\mathbb{1}_{[a,b]}$

Дифференциалы	`dd` и `dx`, `dy`, `dz`, `dt`	$\int x \, \mathrm{d}x \quad \int \rho(x) \, \mathrm{d}\mu(x)$
Частичный дифференциал	`pd`	$\frac{\partial f}{\partial x}$

... и ещё некоторые, смотрите в `src/sym.typ`

Есть нестандартные функции:

- Тригонометрия: $\operatorname{ch} \operatorname{sh} \operatorname{th} \operatorname{tg}$
- Матрицы: $\operatorname{rk} \operatorname{Ker} \operatorname{Im}$
- Мнимые: $\operatorname{Re} \operatorname{Im}$
- Теорвер: $\operatorname{supp} \operatorname{PMF} \operatorname{Bin} \operatorname{Ber} \operatorname{Pois} \operatorname{Geom} \operatorname{Cov}$
- Разное: sign

Если что-то часто встречается в документе, то всегда стоит сделать алиас, чем писать одно и то же сто раз. Если алиас часто используется, то его стоит добавить в шаблон.

Картинки, графики и подобное

Обычные картинки

Кидаете их в папку главы. Поскольку на самом деле при сборке может компилироваться совсем не тот файл, какой ожидаете, относительные пути просто так не работают.

Для этого есть состояние `mb.curDir`. Им можно получить текущую папку главы. Например эта глава скомпилирована из такой папки:

```
/home/i-s-d/doc/ty2/multibook/sample/02-pictures
```

Пользуясь этим можно получить путь к картинке (или к любым другим файлам).

Или воспользоваться уже готовой обёрткой `img(src)`:

Лягушка с overleaf

Она по умолчанию устанавливает размер как максимальный, чтобы картинка была не шире 80% страницы и не была выше 7см.

Но если нужно указать размер вручную, есть аргументы `width` и `height`.

Можно добавить подпись вторым аргументом. Подпись пока что очень примитивно сделана.

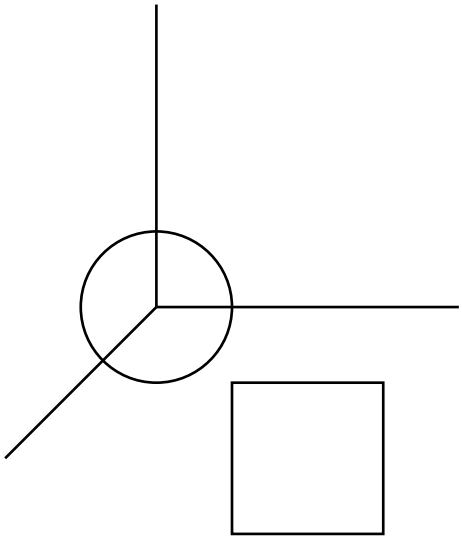
Нумерации нет, поскольку это для конспектов.

Иллюстрации на cetz

Очень полезен `cetz`. Через него можно иллюстрации, в том числе графики. Но если муторно, то лучше просто вставить `svg`-шку и не мучаться.

`cetz` уже импортирован в шаблоне: `canvas` и `draw` доступны глобально. Над `canvas` небольшая обёртка, чтобы выравнивался и не вылетал за пределы странички.

Вторым аргументом можно передать подпись.



Какая-то фигня

Графики

Поверх `cetz`-а я сделал свою библиотечку для построения простых графиков. Все остальные сделаны скорее для научных работ, где нужны красивые подписи для осей и подобное.

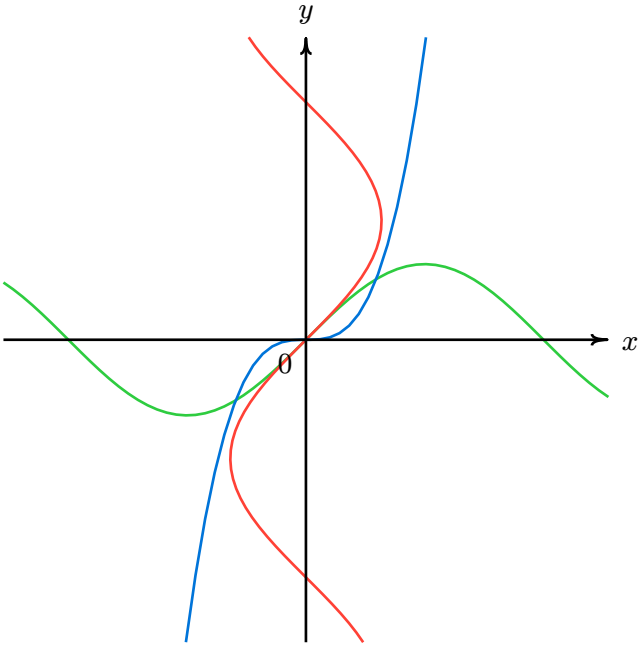
Все её части находятся в `src/plot.typ`, и импортированы как `plot`.

Что есть:

`plot.func()` — строим график функции

- `f` - функция вида `x => y`, например `x => calc.sin(x)`
- `xmin`, `xmax` - диапазон
- `T: false` - если сделать `true`, то `x` и `y` поменяются местами и будет график `y => x`. Диапазон тоже будет по `y`.
- `pts: 64` - количество точек, в которых исполнять функцию
- `c: 1pt + green` - стиль линии
- `ymin: -inf, ymax: inf` - верхняя/нижняя границы по которой обрезать

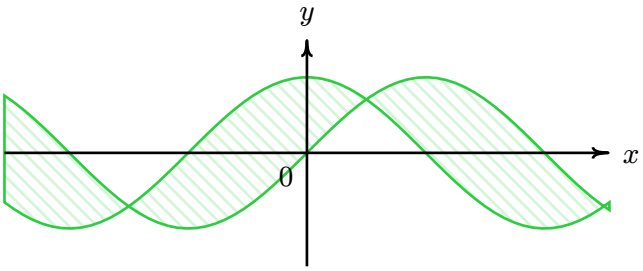
Как можно уже догадаться, эта функция просто берёт `pts` точек по диапазону, выполняет функцию в них и рисует линию.



`plot.func()` в действии

`plot.fill2()` – Заливает площадь между двумя функциями

- `f1`, `f2` - функции `x => y`
- `xmin, xmax` - диапазон
- `T: false` - аналогично с `func()` меняет `x` и `y` местами
- `pts: 64` - число точек на каждую функцию
- `stroke: ...` - стиль линии. Можно сделать `none`.
- `sides: true` - если `true`, то линии и по бокам между функциями. если `false`, то линии только по функциям.
- `fill: ...` - заливка



Площадь между $\sin$ и $\cos$

Для заливки можно использовать `plot.pat_lines(color: ...)`, который выдаёт штрихованную заливку данного цвета.

`plot.axes()` – Рисует оси

- `xmin`, `ymin`, `xmax`, `ymax` - минимумы/максимумы осей
- `extra: false` - увеличивает диапазоны на 10% (иногда удобно)
- `xl: $x$`, `yl: $y$` - подписи X/Y

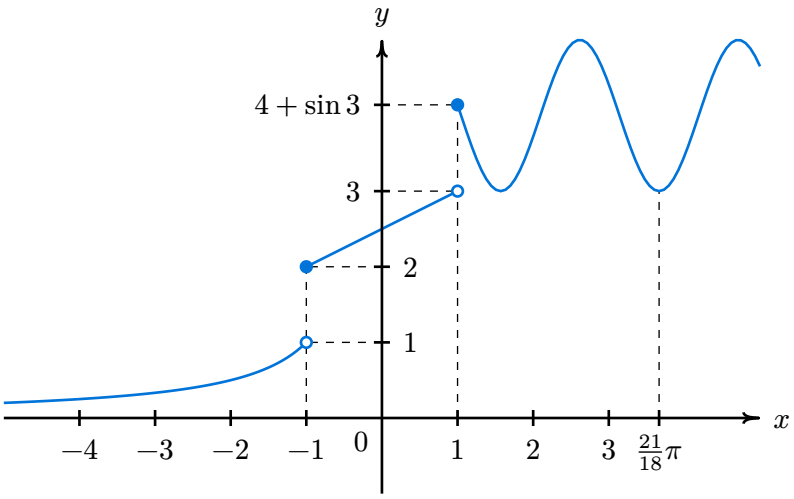
Примеры уже были на других графиках.

`plot.[xy]tick()`, `plot.[xy]perp()` — штрихи и линии

Рисуют штрихи на Ox/Oy по заданной позиции или линии от Ox/Oy к заданной точки. Ещё одним аргументом можно указать подпись.

У штрихов есть аргумент `p`, при `true` подпись выше Ox/правее Oy, при `false` иначе.

У линий есть `c`, чтобы указывать их цвет.



Функция с разрывами

`plot.pt()`, `plot.xpt()` — точки

Первая обычная точка, вторая выколота. Принимают на вход две координаты и `c: ...` - цвет.

См. функцию выше.

3D графики (их нет)

Пока их нет, но было бы неплохо написать плагинчик на Rust, который будет их рисовать.

Пока только SVG или рисованные от руки.

Сборка и внутренности

Как устроен проект

Один конспект – это папка с главами, на каждый семинар / лекцию. В нём имеется корневой `book.typ` и папки с главами, каждая из которых содержит `doc.typ`.

Также в папке лежит `Makefile`, в котором инклюдится `mb.mk`.

В `book.typ` задаются свойства всего конспекта:

```
#import "../mb.typ": * // тут относительный путь до mb.typ

#show: mb.book(
    title: [ Название конспекта ],
    fields: (
        [ Автор ], [ В. Пупкин ],
    ),
    color: blue, // какой вам нравится
    pattern: "diamonds" // один из папки patterns/
)

Какой-то текст, который будет под всеми полями.
```

В `Makefile`:

```
include ../mb.mk # относительный путь до mb.mk
```

И в каждой главе:

```
#import "../../mb.typ": * // тоже отн. путь
#show: mb.chapter(
    title: [ Название семинара ],
    date: [ 4 сентября, или другая дата ],
    msg: [ Какое-то сообщение под названием главы ]
)

Конспект семинара/лекции
```

`Makefile` имеет несколько целей:

- `build/preview.pdf`: превью всего конспекта
- `build/[CHAP_NAME].pdf`: превью главы в папке `[CHAP_NAME]`
- `build/main.pdf`: компиляция релизной версии главы

Для компиляции нужны шрифты `IBM Plex Mono/Serif/Sans`.

Список того, что надо сделать

- Избавиться от контекста: генерим файлы `.typ`, в которых всё инклюдим. `book.typ` чисто задаёт настройки.
- 3D графики: пишем рендерер на расте?